Desenvolvimento de Sistemas Web II

Introdução a Go, com testes, e o domínio do serviço Go

DIM0547 — Turma 01 · Sprint 2 · aula de 26/10

Prof. Fernando · UFRN · 2026.2

Sprint 2

	Data
Conteúdo	hoje: introdução a Go e o Passo 13 · em vídeo, até 30/10: passos 14 a 18
Revisão para a prova	04/11 (presencial) e 09/11 (online)
Acompanhamento online	16/11
Prova escrita	**11/11 (quarta)**, no laboratório
Daily meetings online	23 e 25/11
Entrega final	**30/11, 23:59**

Critério	Peso	Aula
Microsserviço Go	25%	hoje
Contrato Protobuf e integração gRPC	25%	vídeo
Sistema no ar	25%	vídeo
Ambiente e testes (inclui os testes que saíram da Sprint 1)	15%	vídeo
Documentação	10%	—

Onde vamos chegar

- Ao criar uma tarefa, a API pede ao serviço Go que avise por três canais
- O contrato entre os dois é um arquivo `.proto`, versionado no repositório
- Se o serviço Go cair, a tarefa continua sendo criada

Branch `aula`: `./passo.sh 12` é onde a Sprint 1 terminou; hoje vamos até o 13.

Por que um serviço à parte, e em Go

A rubrica pede um serviço com responsabilidade **justificada**. Bons motivos:

Motivo	No exemplo
Trabalho de E/S que espera por terceiros	cada canal espera um servidor externo
Concorrência	os três canais ao mesmo tempo, uma goroutine por canal
Falha isolada	se o serviço cair, a API continua respondendo
Ritmo próprio de implantação	o serviço muda sem republicar a API

Maus motivos: "microsserviço é moderno"; separar um CRUD em dois serviços que dividem o mesmo banco.

> Go cabe bem aqui: goroutines baratas, binário estático, imagem de container pequena e partida rápida.

📖 **Ref.** Effective Go

Roteiro de hoje

Min	Bloco	Exemplo
10	A Sprint 2 e por que um serviço em Go	a figura do sistema
10	Ferramentas e tipos	`go mod`, `go run`, `go test`
20	Olá, Mundo com testes; funções como valores	`2-hello-world`
15	Métodos e interfaces	`3-metodos-interfaces`
15	Ponteiros e erros	`4-ponteiros-erros`
10	Injeção de dependência	`5-injecao-de-dependencia`
20	Passo 13: o domínio do serviço de avisos	`exemplos/avisos-go`

Os exemplos de Go vêm do repositório do Curso básico de Go, que segue o livro *Aprenda Go com Testes*:

Parte 1

Go com testes

As ferramentas

Termo	O que é
módulo	o projeto: uma pasta com `go.mod`, que declara o nome e as dependências
pacote	uma pasta de arquivos `.go` com o mesmo `package`; `main` gera um executável
teste	uma função `TestXxx(t *testing.T)` num arquivo `_test.go`, ao lado do código

O teste vem com a linguagem: não há biblioteca de teste para escolher nem configurar.

Tipagem: estática, forte e com inferência

	Go	Java	Python
Quando os tipos são verificados	compilação (estática)	compilação	execução (dinâmica)
Conversão implícita entre tipos	não (forte)	em alguns casos (`int` → `double`)	não, mas os tipos só aparecem na execução
Precisa escrever o tipo?	não, quando o compilador deduz	`var` desde o Java 10	não

> Inferência não é tipagem dinâmica: `idade := 20` tem tipo `int` para sempre.

📖 **Ref.** Curso básico de Go, `2-hello-world/slides/slides-tipos-em-go.md`

Olá, Mundo, com o teste primeiro

O ciclo do livro: **vermelho** (um teste que falha) → **verde** (o mínimo para passar) → **refatorar** (melhorar com os testes verdes).

```
```

Na tela: `cd 2-hello-world && go test -v`; quebrar um caso e ler a mensagem; `go test -run 'TestHello/região'`.

Funções como valores: *functional options*

- Go não tem sobrecarga nem parâmetros com valor padrão: as opções resolvem as duas coisas
- `WithName` devolve uma *closure*: uma função que guarda o `name` recebido
- `&cfg` passa o endereço da configuração; é por ele que a opção consegue alterá-la (mais adiante, ponteiros)

📖 **Ref.** Curso básico de Go, `3-metodos-interfaces/slides/polimorfismo-em-go.md` (Overloading × Functional Options)

Métodos e interfaces: satisfação implícita

```

```

```

```

Nenhum `implements`: quem tem o método `Area() float64` é uma `Forma`, e o compilador confere.

📖 **Ref.** Curso básico de Go, `3-metodos-interfaces/slides/polimorfismo-em-go.md`

Interfaces em Java e em Go

Java

Go

A classe declara que implementa. A interface precisa existir antes, e a classe depende dela.

O tipo não sabe da interface. Quem usa declara a interface de que precisa.

Consequência: em Go, a interface fica **com quem usa**, e costuma ter um ou dois métodos. É o que vamos ver em `io.Writer` e no serviço de avisos (`Canal`).

Ponteiros: o receptor por valor recebe uma cópia

```

```

O que acontece sem o `*`:

```

```

Receptor	Quando usar
por valor `(c Carteira)`	o método só lê; o tipo é pequeno
por ponteiro `(c *Carteira)`	o método altera o estado, ou o tipo é grande

Na tela: em `4-ponteiros-erros`, tirar o `*` de `Depositar` e rodar `go test -v`.

Erros: valores, com tipo próprio

```

```

```

```

- `error` é uma interface com um método, `Error() string`; o erro aparece na assinatura e é tratado por quem chamou

📖 **Ref.** Go — errors

Injeção de dependência: `io.Writer`

Quem chama passa	E a saudação vai para
`os.Stdout`	o terminal
`&bytes.Buffer{}`	a memória, para o teste conferir
o `http.ResponseWriter` de um servidor HTTP	a resposta da requisição

Depender de uma interface, e não de um destino concreto, é o que torna a função testável: a ideia das portas da API de Tarefas.

Parte 2 — Passo 13

O domínio do serviço de avisos

O domínio: uma interface pequena e o Avisador

- `CanalSimulado` tem `Nome()` e `Enviar(...)`: é um `Canal`, sem declarar nada
- O pacote `avisos` é o domínio: não importa gRPC, HTTP nem banco
- Como o `io.Writer`: uma interface pequena, declarada por quem usa, com um método que pode falhar

Concorrência: uma goroutine por canal

- Cada goroutine escreve só na sua posição da fatia: não há disputa
- Canais de 400, 50 e 150 ms: em sequência, 600 ms; ao mesmo tempo, **400 ms**, o do mais lento

📖 **Ref.** Go — sync.WaitGroup

`context`: prazo e cancelamento

```

```

```

```

- Toda função que pode esperar recebe `ctx context.Context` como **primeiro** parâmetro
- O prazo atravessa as camadas: quem chama decide quanto está disposto a esperar

📖 **Ref.** Go — context

Erros com nome, e a demonstração

	Quando usar	Como conferir
erro com nome (`ErrAvisoInvalido`)	quem chama só precisa saber **qual** erro foi	`errors.Is(err, ErrAvisoInvalido)`
tipo de erro (`ErroSaldoInsuficiente`)	quem chama precisa de **dados** do erro	`errors.As(err, &alvo)`

Demonstração

No vídeo (até 30/10)

Passo	Assunto
14	O contrato: Protocol Buffers e Buf
15	O servidor gRPC em Go: status, prazo, health e reflection
16	A API chama o Go: cliente gRPC no Ktor e no Quarkus
17	Tudo em containers: Dockerfiles, health checks e `docker compose`
18	Implantação: imagens no GHCR, migração por comando, Render e Neon

Até lá, no projeto do grupo:

- fazer os exercícios do curso de Go que faltarem (`2-hello-world` a `5-injecao-de-dependencia`)
- escolher a responsabilidade do serviço Go e escrever a justificativa (uma ADR)
- escrever o domínio do serviço, com testes, como no Passo 13

Onde estudar depois

- Curso básico de Go (vídeos do professor) e o repositório dos exemplos
- Aprenda Go com Testes: os capítulos de Olá, Mundo; estruturas, métodos e interfaces; ponteiros e erros; injeção de dependência; concorrência e select
- Learn Go with Tests — Context (em inglês)
- A Tour of Go · Effective Go
- Leitura da sprint: `leituras/web2-s2.md`, capítulos 1 a 3
- Passo a passo: `exemplos/avisos-go/PASSOS.md` · branch `aula`, tag `passo-13`